Day 43 - Multiple Linear Regression

Introduction

In the previous day (Day41), I implemented **Simple Linear Regression** using one independent variable. On this day, I extended the concept to **Multiple Linear Regression (MLR)** where we use **two or more independent variables** to predict a dependent variable.

This notebook covers:

1. Understanding how Multiple Linear Regression works
2. Data preprocessing using `pd.get_dummies()`
3. Applying Linear Regression with scikit-learn
4. Interpreting model parameters (coefficients & intercept)
5. Evaluating model performance
6. Using **OLS** (Ordinary Least Squares) from `statsmodels.api` to get detailed statistics
7. Performing **Backward Elimination** using p-values to refine the model
8. Learning about APIs in ML and the role of `statsmodels.api`

The practical coding was done in **Spyder IDE** for execution, while this Jupyter Notebook documents the full process with explanations.

What is Multiple Linear Regression?

Multiple Linear Regression (MLR) is an extension of Simple Linear Regression.

- In **SLR** → we use **one independent variable (X)** to predict the dependent variable (y).
- In **MLR** → we use **two or more independent variables (X_1, X_2, ..., X_n)** to predict y.

Equation: [y = b_0 + b_1x_1 + b_2x_2 + ... + b_nx_n]

- **b0 (intercept):** value of y when all X = 0
- **b1, b2, ... (coefficients):** how much y changes when that variable increases by 1 unit, keeping others constant

Example: Predicting House Price using → size, number of bedrooms, location, etc.

Import Libraries

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import statsmodels.api as sm
import pickle
```

Import Dataset

Dataset Information

For this exercise, I used a dataset that contains information about a company's **startups and their profits**. The goal is to build a regression model that predicts **Profit** based on different independent variables.

Features:

- **DigitalMarketing** → Money invested in marketing campaigns
- **Research** → Money spent on research and development
- **Promotion** → Money spent on promotional activities
- **State** → The state in which the startup operates (categorical feature)

Target:

- **Profit** → The company's profit (dependent variable to predict)

Why this dataset?

This dataset is a classic example for Multiple Linear Regression because:

- It contains both **numeric features** (spendings) and a **categorical feature** (state)
- The relationship between profit and predictors is approximately linear
- It allows us to practice **encoding categorical variables**, fitting regression models, and performing **feature selection** (Backward Elimination).

In [2]:
```
dataset = pd.read_csv(r"C:\Users\Arman\Downloads\dataset\Investment.csv")
```

In [3]:
```
dataset.head(5)
```

Out[3]:

	DigitalMarketing	Promotion	Research	State	Profit
0	165349.20	136897.80	471784.10	Hyderabad	192261.83
1	162597.70	151377.59	443898.53	Bangalore	191792.06
2	153441.51	101145.55	407934.54	Chennai	191050.39
3	144372.41	118671.85	383199.62	Hyderabad	182901.99
4	142107.34	91391.77	366168.42	Chennai	166187.94

In [4]:
```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   DigitalMarketing  50 non-null     float64
 1   Promotion         50 non-null     float64
 2   Research          50 non-null     float64
 3   State             50 non-null     object 
 4   Profit            50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
```

In [5]:
```
dataset.columns
```

Out[5]:
```
Index(['DigitalMarketing', 'Promotion', 'Research', 'State', 'Profit'], dtype='object')
```

In [6]:
```
dataset.isnull().sum()
```

Out[6]:
```
DigitalMarketing    0
Promotion           0
Research            0
State               0
Profit              0
dtype: int64
```

Separating Independent & Dependent Variables

In [7]:
```
X = dataset.iloc[:, :-1]
y = dataset.iloc[:, 4]
```

Data Preprocessing

In this dataset, some features were **categorical** (e.g., "State").
Since regression models require **numerical inputs**, we converted categorical columns into numeric using **dummy variables**:

- Used `pd.get_dummies()` to create dummy variables (0/1) for each category.
- Example: "State" column → "Bangalore", "Chennai", etc.
- One column is usually dropped automatically (to avoid the **dummy variable trap**) since it can be derived from the others.

This method is called **One-Hot Encoding**, and `pandas.get_dummies()` makes it easy to apply.

Encode Categorical Data

In [8]:
```
X = pd.get_dummies(X,dtype=int)
```

In [9]:
```
X
```

Out[9]:

	DigitalMarketing	Promotion	Research	State_Bangalore	State_Chennai	State_Hyderabad
0	165349.20	136897.80	471784.10	0	0	1
1	162597.70	151377.59	443898.53	1	0	0
2	153441.51	101145.55	407934.54	0	1	0
3	144372.41	118671.85	383199.62	0	0	1
4	142107.34	91391.77	366168.42	0	1	0
5	131876.90	99814.71	362861.36	0	0	1
6	134615.46	147198.87	127716.82	1	0	0
7	130298.13	145530.06	323876.68	0	1	0
8	120542.52	148718.95	311613.29	0	0	1
9	123334.88	108679.17	304981.62	1	0	0
10	101913.08	110594.11	229160.95	0	1	0
11	100671.96	91790.61	249744.55	1	0	0
12	93863.75	127320.38	249839.44	0	1	0
13	91992.39	135495.07	252664.93	1	0	0
14	119943.24	156547.42	256512.92	0	1	0
15	114523.61	122616.84	261776.23	0	0	1
16	78013.11	121597.55	264346.06	1	0	0
17	94657.16	145077.58	282574.31	0	0	1
18	91749.16	114175.79	294919.57	0	1	0
19	86419.70	153514.11	0.00	0	0	1
20	76253.86	113867.30	298664.47	1	0	0
21	78389.47	153773.43	299737.29	0	0	1
22	73994.56	122782.75	303319.26	0	1	0
23	67532.53	105751.03	304768.73	0	1	0
24	77044.01	99281.34	140574.81	0	0	1
25	64664.71	139553.16	137962.62	1	0	0
26	75328.87	144135.98	134050.07	0	1	0
27	72107.60	127864.55	353183.81	0	0	1
28	66051.52	182645.56	118148.20	0	1	0
29	65605.48	153032.06	107138.38	0	0	1
30	61994.48	115641.28	91131.24	0	1	0
31	61136.38	152701.92	88218.23	0	0	1
32	63408.86	129219.61	46085.25	1	0	0
33	55493.95	103057.49	214634.81	0	1	0
34	46426.07	157693.92	210797.67	1	0	0
35	46014.02	85047.44	205517.64	0	0	1
36	28663.76	127056.21	201126.82	0	1	0
37	44069.95	51283.14	197029.42	1	0	0

	DigitalMarketing	Promotion	Research	State_Bangalore	State_Chennai	State_Hyderabad
38	20229.59	65947.93	185265.10	0	0	1
39	38558.51	82982.09	174999.30	1	0	0
40	28754.33	118546.05	172795.67	1	0	0
41	27892.92	84710.77	164470.71	0	1	0
42	23640.93	96189.63	148001.11	1	0	0
43	15505.73	127382.30	35534.17	0	0	1
44	22177.74	154806.14	28334.72	1	0	0
45	1000.23	124153.04	1903.93	0	0	1
46	1315.46	115816.21	297114.46	0	1	0
47	0.00	135426.92	0.00	1	0	0
48	542.05	51743.15	0.00	0	0	1
49	0.00	116983.80	45173.06	1	0	0

Applying Multiple Linear Regression

Steps followed:

1. Split the dataset into **training** and **test** sets
2. Fit the **LinearRegression** model from scikit-learn on training data
3. Predict on the test data
4. Compare predicted vs actual values

This gives us a baseline performance of the regression model.

Split the dataset into Training & Testing

In [10]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

Train the model

In [11]:
```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train,y_train)
```

Out[11]:
```
▾ LinearRegression
LinearRegression()
```

Predict the test set & Compare

In [12]:
```
y_pred = regressor.predict(X_test)
y_pred
```

Out[12]:
```
array([103015.20159796, 132582.27760816, 132447.73845174,  71976.09851258,
       178537.48221055, 116161.24230165,  67851.69209676,  98791.73374687,
       113969.43533012, 167921.0656955 ])
```

In [13]:
```
comparison = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print(comparison)
```

```
        Actual      Predicted
28  103282.38  103015.201598
11  144259.40  132582.277608
10  146121.95  132447.738452
41   77798.83   71976.098513
2   191050.39  178537.482211
27  105008.31  116161.242302
38   81229.06   67851.692097
31   97483.56   98791.733747
22  110352.25  113969.435330
4   166187.94  167921.065696
```

Model Parameters

In [14]:
```
m = regressor.coef_
print("Slope(m):", m)
```

```
Slope(m): [ 7.73467193e-01  3.28845975e-02  3.66100259e-02  8.66383692e+01
 -8.72645791e+02  7.86007422e+02]
```

In [15]:
```
c = regressor.intercept_
print("Intercept(c):",c)
```

```
Intercept(c): 42467.52924855311
```

Model Performance

Training Score

In [16]:
```
bias = regressor.score(X_train, y_train)
bias
```

Out[16]:
```
0.9501847627493607
```

Testing Score

In [17]:
```
variance = regressor.score(X_test, y_test)
variance
```

Out[17]:
```
0.9347068473282424
```

Ordinary Least Squares (OLS)

While scikit-learn gives us predictions and coefficients, it does **not provide detailed statistics**.
For deeper insights, we use **OLS (Ordinary Least Squares)** from `statsmodels.api`.

OLS gives us:

- Coefficients with **p-values** (significance of variables)
- R^2 and Adjusted R^2
- F-statistic (overall model significance)
- Confidence intervals

This helps us understand which variables actually matter in predicting y.

Add Constant for OLS

OLS (Ordinary Least Squares) from statsmodels requires a constant column to represent the intercept.

Add Constant Column

In [18]:
```
X = np.append(arr=np.full((50,1), 42467).astype(int), values=X, axis=1)
```

In [19]:
```
pd.DataFrame(X).head()
```

Out[19]:

	0	1	2	3	4	5	6
0	42467.0	165349.20	136897.80	471784.10	0.0	0.0	1.0
1	42467.0	162597.70	151377.59	443898.53	1.0	0.0	0.0
2	42467.0	153441.51	101145.55	407934.54	0.0	1.0	0.0
3	42467.0	144372.41	118671.85	383199.62	0.0	0.0	1.0
4	42467.0	142107.34	91391.77	366168.42	0.0	1.0	0.0

Feature Selection: Backward Elimination

Not all features improve the model. Some may be **insignificant**.
We use **Backward Elimination**:

1. Fit model with all features
2. Look at **p-values**
3. Remove the feature with the **highest p-value** (if p > 0.05)
4. Refit the model
5. Repeat until all features are significant

This improves the model by keeping only important predictors.

API & statsmodels.api

- **API (Application Programming Interface):**
  In ML, an API lets us interact with libraries (like scikit-learn or statsmodels) through simple functions.
  Example: `sm.OLS(y, X).fit()` is an API call that runs regression for us.
- **statsmodels.api:**
  A Python library used for statistical analysis.
  - More detailed than scikit-learn for regression
  - Provides p-values, confidence intervals, and other statistical tests
  - Very useful for research and feature selection

In [20]:
```
import statsmodels.api as sm
X_opt = X[:,[0,1,2,3,4,5]]
#OrdinaryLeastSquares
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()
```

Out[20]:

OLS Regression Results

Dep. Variable:	Profit	**R-squared:**	0.951
Model:	OLS	**Adj. R-squared:**	0.945
Method:	Least Squares	**F-statistic:**	169.9
Date:	Tue, 26 Aug 2025	**Prob (F-statistic):**	1.34e-27
Time:	20:21:13	**Log-Likelihood:**	-525.38
No. Observations:	50	**AIC:**	1063.
Df Residuals:	44	**BIC:**	1074.
Df Model:	5		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	1.1794	0.164	7.204	0.000	0.849	1.509
x1	0.8060	0.046	17.369	0.000	0.712	0.900
x2	-0.0270	0.052	-0.517	0.608	-0.132	0.078
x3	0.0270	0.017	1.574	0.123	-0.008	0.062
x4	41.8870	3256.039	0.013	0.990	-6520.229	6604.003
x5	240.6758	3338.857	0.072	0.943	-6488.349	6969.701

Omnibus:	14.782	**Durbin-Watson:**	1.283
Prob(Omnibus):	0.001	**Jarque-Bera (JB):**	21.266
Skew:	-0.948	**Prob(JB):**	2.41e-05
Kurtosis:	5.572	**Cond. No.**	8.45e+05

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 8.45e+05. This might indicate that there are
strong multicollinearity or other numerical problems.

Remove Feature with Highest p-value

Keep repeating this by removing the feature with the highest p-value above 0.05 until all remaining features are significant.

In [21]:

```
import statsmodels.api as sm
X_opt = X[:,[0,1,2,3,5]]
#OrdinaryLeastSquares
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()
```

Out[21]:

OLS Regression Results

Dep. Variable:	Profit	**R-squared:**	0.951
Model:	OLS	**Adj. R-squared:**	0.946
Method:	Least Squares	**F-statistic:**	217.2
Date:	Tue, 26 Aug 2025	**Prob (F-statistic):**	8.49e-29
Time:	20:21:13	**Log-Likelihood:**	-525.38
No. Observations:	50	**AIC:**	1061.
Df Residuals:	45	**BIC:**	1070.
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	1.1799	0.157	7.537	0.000	0.865	1.495
x1	0.8060	0.046	17.606	0.000	0.714	0.898
x2	-0.0270	0.052	-0.523	0.604	-0.131	0.077
x3	0.0270	0.017	1.592	0.118	-0.007	0.061
x4	220.1585	2900.536	0.076	0.940	-5621.821	6062.138

Omnibus:	14.758	**Durbin-Watson:**	1.282
Prob(Omnibus):	0.001	**Jarque-Bera (JB):**	21.172
Skew:	-0.948	**Prob(JB):**	2.53e-05
Kurtosis:	5.563	**Cond. No.**	6.18e+05

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 6.18e+05. This might indicate that there are
strong multicollinearity or other numerical problems.

Final Model after Elimination

In [22]:

```
import statsmodels.api as sm
X_opt = X[:,[0,1]]
#OrdinaryLeastSquares
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()
```

Out[22]:

OLS Regression Results

Dep. Variable:	Profit	**R-squared:**	0.947
Model:	OLS	**Adj. R-squared:**	0.945
Method:	Least Squares	**F-statistic:**	849.8
Date:	Tue, 26 Aug 2025	**Prob (F-statistic):**	3.50e-32
Time:	20:21:13	**Log-Likelihood:**	-527.44
No. Observations:	50	**AIC:**	1059.
Df Residuals:	48	**BIC:**	1063.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	1.1546	0.060	19.320	0.000	1.034	1.275
x1	0.8543	0.029	29.151	0.000	0.795	0.913

Omnibus:	13.727	**Durbin-Watson:**	1.116
Prob(Omnibus):	0.001	**Jarque-Bera (JB):**	18.536
Skew:	-0.911	**Prob(JB):**	9.44e-05
Kurtosis:	5.361	**Cond. No.**	4.60

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Save the model using pickle

In [24]:
```
# Save the trained model to disk
filename = 'multiple_linear_regression_model.pkl'
with open(filename, 'wb') as file:
    pickle.dump(regressor, file)
print("Model has been pickled and saved as multiple_linear_regression_model.pkl")
```

```
Model has been pickled and saved as multiple_linear_regression_model.pkl
```

Summary — Day43: Multiple Linear Regression

- Learned the concept of **Multiple Linear Regression** and its equation
- Preprocessed data using `pd.get_dummies()` for categorical encoding
- Trained and tested the model using scikit-learn
- Understood **Model Parameters (coefficients & intercept)** and built the regression equation
- Evaluated model performance
- Used **statsmodels.api (OLS)** to obtain detailed regression statistics
- Applied **Backward Elimination** with p-values for feature selection
- Learned the idea of **APIs** in machine learning libraries

Key Takeaways

- MLR handles multiple predictors simultaneously

- Encoding categorical data is necessary before applying ML models
- **OLS** gives deeper statistical insights compared to scikit-learn
- **Backward Elimination** helps simplify the model by keeping only significant features
- This forms the base for building **practical regression projects**